

Lab 4: Object-Oriented Programming in Python

1 Introduction to Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm that uses the concept of objects to design and develop applications. It allows us to encapsulate data and functions that manipulate the data into a single unit called a class. This helps in organizing code and making it more modular, reusable, and easier to maintain.

2 Key Concepts in OOP

2.1 Class: A class is a blueprint for creating objects (instances). It defines the attributes and methods that the objects created from the class will have.

2.2 Object: An object is an instance of a class. When a class is defined, no memory is allocated until an object of that class is created.

2.3 Constructor: A constructor is a special method in a class that gets called automatically when an object of the class is created. In Python, the constructor method is `__init__()`.

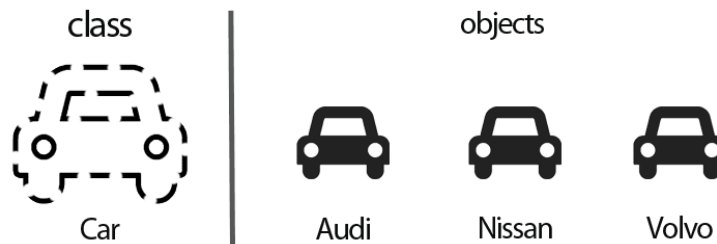


Figure 1: Class of car

2.3.1 Example:

```
1 class Car:
2     def __init__(self, make, model, year):
3         self.make = make
4         self.model = model
5         self.year = year
6
7     def display_info(self):
8         print(f"Car: {self.year} {self.make} {self.model}")
9
10 # Creating an object of the Car class
11 my_car = Car("Toyota", "Corolla", 2020)
12 my_car.display_info()
```

Listing 1: Class with Constructor and Attributes

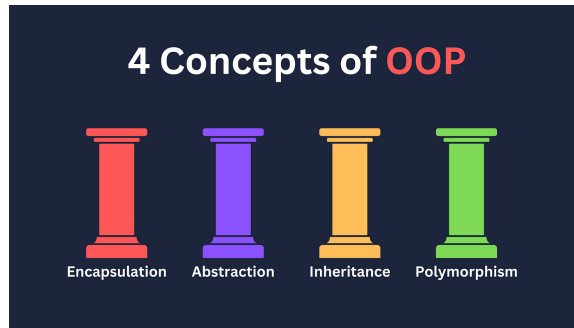


Figure 2: pillars of oops

2.4 Encapsulation: Encapsulation is the practice of wrapping data (attributes) and methods (functions) into a single unit or class. It also involves restricting direct access to some of the object's components, which helps in safeguarding the integrity of the data.

3 ATM Example without Encapsulation

Here is an example of an ATM class without any encapsulation. This class allows a user to create a PIN, deposit money, withdraw money, and check their balance.

```

1 class ATM:
2     def __init__(self):
3         self.pin = ""
4         self.balance = 0
5         self.menu()
6
7     def menu(self):
8         user_input = input("""Select any 1 option:
9                             1) Create User PIN
10                            2) Withdraw
11                            3) Deposit
12                            4) Check Balance
13                            5) Exit
14                            """)
15         if user_input == '1':
16             self.create_pin()
17         elif user_input == '2':
18             self.withdraw()
19         elif user_input == '3':
20             self.deposit()
21         elif user_input == '4':
22             self.check_balance()
23         else:
24             print("Bye bye")
25
26     def create_pin(self):
27         self.pin = input('Enter new PIN: ') # Keep PIN as string for easier
28         comparison
29         print("PIN set successfully")
30
31     def deposit(self):
32         temp = input('Enter your PIN: ')
33         if temp == self.pin:
34             amount = int(input("Enter the amount you want to deposit: "))
35             self.balance += amount
36             print("Your latest balance is: ", self.balance)
37         else:

```

```

37         print("Invalid PIN")
38
39     def withdraw(self):
40         temp = input('Enter your PIN: ')
41         if temp == self.pin:
42             amount = int(input("Enter the amount you want to withdraw: "))
43             if amount <= self.balance:
44                 self.balance -= amount
45                 print("Withdrawal successful. Your latest balance is: ", self.balance)
46             else:
47                 print("You don't have sufficient funds")
48         else:
49             print("Invalid PIN")
50
51     def check_balance(self):
52         temp = input('Enter your PIN: ')
53         if temp == self.pin:
54             print("Your balance is: ", self.balance)
55         else:
56             print("Invalid PIN")
57
58 # Creating an object of ATM class
59 atm = ATM()

```

Listing 2: ATM Class without Encapsulation

3.1 Explanation::

- The ATM class has attributes `pin` and `balance` that store the user's PIN and account balance, respectively.
- The constructor method `__init__()` initializes these attributes and calls the `menu()` method to display the options to the user.
- The methods `create_pin()`, `deposit()`, `withdraw()`, and `check_balance()` provide the functionality to interact with the ATM.

4 ATM Example with Encapsulation

To encapsulate the `pin` and `balance` attributes, we can make them private by prefixing them with a double underscore `__`. This restricts direct access to these attributes from outside the class, allowing them to be accessed only through the class methods.

```

1 class ATM:
2     def __init__(self):
3         self.__pin = ""
4         self.__balance = 0
5         self.menu()
6
7     def menu(self):
8         user_input = input("""Select any 1 option:
9                             1) Create User PIN
10                            2) Withdraw
11                            3) Deposit
12                            4) Check Balance
13                            5) Exit
14                            """)
15         if user_input == '1':
16             self.create_pin()
17         elif user_input == '2':

```

```

18         self.withdraw()
19     elif user_input == '3':
20         self.deposit()
21     elif user_input == '4':
22         self.check_balance()
23     else:
24         print("Bye bye")
25
26     def create_pin(self):
27         self.__pin = input('Enter new PIN: ')
28         print("PIN set successfully")
29
30     def deposit(self):
31         temp = input('Enter your PIN: ')
32         if temp == self.__pin:
33             amount = int(input("Enter the amount you want to deposit: "))
34             self.__balance += amount
35             print("Your latest balance is: ", self.__balance)
36         else:
37             print("Invalid PIN")
38
39     def withdraw(self):
40         temp = input('Enter your PIN: ')
41         if temp == self.__pin:
42             amount = int(input("Enter the amount you want to withdraw: "))
43             if amount <= self.__balance:
44                 self.__balance -= amount
45                 print("Withdrawal successful. Your latest balance is: ", self.
46                     __balance)
47             else:
48                 print("You don't have sufficient funds")
49         else:
50             print("Invalid PIN")
51
52     def check_balance(self):
53         temp = input('Enter your PIN: ')
54         if temp == self.__pin:
55             print("Your balance is: ", self.__balance)
56         else:
57             print("Invalid PIN")
58
59 # Creating an object of ATM class
60 atm = ATM()

```

Listing 3: ATM Class with Encapsulation

4.1 Explanation::

- The attributes `pin` and `balance` are now private, indicated by the double underscore prefix (`--`).
- These private attributes can no longer be accessed directly from outside the class, providing better security for sensitive data like the PIN and balance.
- The methods `create_pin()`, `deposit()`, `withdraw()`, and `check_balance()` allow controlled access to these private attributes.

5 Conclusion

In this lab, we covered the basic concepts of Object-Oriented Programming (OOP) in Python, including classes, objects, constructors, and encapsulation. We demonstrated these concepts using

PREPARED BY: Sahil Wagh (M.Tech)

an ATM system example, illustrating how encapsulation can protect sensitive data and ensure secure interactions with the system.